Simple Linear Regression Program Using Gradient Descent

Dr. Srikanth Thota

Introduction

Purpose

This document describes the design and implementation of a simple linear regression program that fits a line to given data points using gradient descent.

Scope

The program fits a line $y = b_1x + b_0$ to a small dataset by minimizing the mean squared error (MSE).

Definitions, Acronyms, and Abbreviations

- **MSE:** Mean Squared Error
- **Gradient Descent:** An optimization algorithm to minimize the cost function iteratively.
- b_0: Intercept of the regression line
- b_1: Slope of the regression line

System Overview

The system takes input arrays X and y, initializes parameters b_0 and b_1, and performs iterative updates using gradient descent to minimize the prediction error (MSE). It outputs the fitted parameters and visualizes the data and convergence process.

System Features

Input

- X: Independent variable vector $\{2, 3, 4, 5\}$
- y: Dependent variable vector $\{4, 6, 8, 10\}$

Output

- Fitted parameters b_0, b_1
- Scatter plot of original data and regression line
- Plot of loss (MSE) per iteration

External Interface Requirements

Hardware Interfaces

No specific hardware requirements other than a typical computer capable of running Python and matplotlib.

Software Interfaces

- Python 3.x
- NumPy for numerical operations
- Matplotlib for data visualization

System Design

Data Initialization

$$X = \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \end{bmatrix}, \quad y = \begin{bmatrix} 4 \\ 6 \\ 8 \\ 10 \end{bmatrix}$$

Flattened arrays are used during computation.

Parameter Initialization

Parameter	Initial Value	Description
b_0	0.0	Intercept
b_1	0.0	Slope
Learning rate α	0.1	Gradient descent step size
Number of iterations	260	Number of updates
m	4	Number of data points

Algorithm Steps

For each iteration $i = 1, 2, \ldots, 260$ perform:

1. Compute predicted values:
$$\hat{y} = b_0 + b_1 \times X$$
2. Calculate gradients:
$$\frac{\partial \text{Loss}}{\partial b_0} = db_0 = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i)$$
$$\frac{\partial \text{Loss}}{\partial b_1} = db_1 = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i) x_i$$
3. Update parameters:
$$b_0 := b_0 - \alpha \cdot db_0$$
$$b_1 := b_1 - \alpha \cdot db_1$$
4. Calculate and store loss:
$$\text{Loss} = \frac{1}{2m} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2$$

Implementation

The program iterates over 260 updates applying gradient descent, updating b_0, b_1, and recording loss values for plotting the convergence graph.

Testing

- Verify initial data printing and parameter initialization.
- Confirm loss decreases monotonically.
- Validate b_0, b_1 values approximate the expected relationship $y = 2x$.
- Inspect plots of data fit and loss convergence.

Limitations and Assumptions

- Dataset is fixed and small.
- No input validation or error handling.
- Learning rate and iterations are fixed constants.
- Only simple linear regression (univariate).
- No regularization or advanced optimization techniques.

Future Enhancements

- Modularize code into functions or classes.

Appendix: Code Snippet

```
m = len(y)
b0, b1 = 0.0, 0.0
n_iter = 260
learn_rate = 0.1
loss_hist = []

for i in range(n_iter):
y_pred = b0 + b1 * X.flatten()
db0 = (1/m) * np.sum(y_pred - y.flatten())
db1 = (1/m) * np.sum((y_pred - y.flatten()) * X.flatten())
b0 -= learn_rate * db0
b1 -= learn_rate * db1
loss = np.sum((y_pred - y.flatten()) ** 2) / (2 * m)
loss_hist.append(loss)
```